

CS Rigor Enhancement Analysis for Zarailink

Executive Summary

Zarailink is a Pakistani agricultural trade intelligence SaaS platform that has achieved an impressive MVP scope: a 4-stage search pipeline (NLP parsing → semantic matching → ORM aggregation → Learn-to-Rank ensemble), a company graph with Node2Vec embeddings, HDBSCAN clustering, and a full-featured React frontend. This is far above the typical CRUD final-year project.

However, the rigor gaps are specific and identifiable. The search pipeline's Stage 1 (QueryInterpreter) relies entirely on regex and keyword-counting heuristics — no learned model, no formal grammar, no evaluation framework. The semantic search (Stage 2) uses a pretrained SBERT model as a black box with brute-force cosine similarity over a pickle file — no custom fine-tuning, no approximate nearest neighbor indexing, no retrieval evaluation metrics. The Node2Vec embeddings exist but aren't connected to a proper GNN-based link prediction system. And critically, the `buyer`/`seller` fields in the Transaction table are raw, unnormalized text strings — the single largest data quality problem in the system — with no entity resolution pipeline to canonicalize them.

The proposed enhancements below target these exact gaps. Each one builds on existing architecture, requires implementing non-trivial algorithms from scratch or with deep customization, and produces measurable, evaluable outcomes that demonstrate genuine CS learning.

Current Architecture Analysis

Components With Existing CS Depth

The project already demonstrates non-trivial CS work in several areas:

- **4-stage search pipeline architecture:** The decomposition into parse → match → aggregate → rank is a well-designed information retrieval pipeline. The family classification system (F1–F9) shows genuine domain modeling.
- **LightGBM LambdaRank (Stage 4):** Using a listwise learning-to-rank objective for supplier ranking is a legitimate ML technique, even if currently trained on pseudo-labels.
- **Node2Vec graph embeddings:** Computing random-walk-based embeddings on a buyer-seller transaction graph and using HDBSCAN clustering for company segmentation demonstrates graph ML awareness.
- **Hybrid matching (Stage 2):** Combining keyword, ProductItem propagation, SBERT semantic, and fuzzy matching shows multi-signal retrieval thinking.

Rigor Gaps

Component	Current State	Why It Lacks CS Rigor
Stage 1: QueryInterpreter	Regex + keyword counting	No formal grammar, no learned classifier, no evaluation. A hand-coded <code>if/elif</code> chain is not academically defensible as "NLP"
Stage 2: Semantic Search	Brute-force cosine over pickle file	Uses pretrained model as black box; O(n) linear scan; no ANN index; no fine-tuning; no retrieval evaluation (P@K, NDCG)
Entity Resolution	None — <code>buyer</code> / <code>seller</code> are raw text	"Muller & Phipps", "MULLER AND PHIPPS PVT LTD", "M&P Pakistan" are treated as different entities. This is the #1 data quality problem
GNN Link Prediction	Node2Vec only; no actual GNN	Node2Vec is a shallow embedding; no message-passing GNN, no link prediction task, no evaluation
Anomaly Detection	None	No pipeline to detect price outliers, volume spikes, or suspicious transactions in trade data
LTR Training Loop	Pseudo-labels only	No click-through integration, no online learning, no proper NDCG evaluation with held-out sets

Research Findings

Domain-Specific Research

The project sits at the intersection of **information retrieval**, **graph machine learning**, **entity resolution**, and **time-series anomaly detection** — all active research areas with rich academic literature.

Key insight: The raw `buyer`/`seller` text fields in the Transaction model are the fundamental data integrity challenge. Every downstream component (aggregation, graph embeddings, company profiles) suffers from this. Entity resolution is not just a feature — it's a precondition for correctness.

Citation Index

1. Fellegi, I.P. & Sunter, A.B. (1969). "A Theory for Record Linkage." *Journal of the American Statistical Association*, 64(328), 1183–1210.
2. Binette, O. & Steorts, R.C. (2022). "(Almost) All of Entity Resolution." *Science Advances*, 8(12), eabi8021.
3. Malkov, Y. & Yashunin, D. (2020). "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs." *IEEE TPAMI*, 42(4), 824–836.
4. Zhang, M. & Chen, Y. (2018). "Link Prediction Based on Graph Neural Networks." *NeurIPS 2018*, 5165–5175.
5. Kipf, T.N. & Welling, M. (2017). "Semi-Supervised Classification with Graph Convolutional Networks." *ICLR 2017*.
6. Schmidl, S. et al. (2022). "Anomaly Detection in Time Series: A Comprehensive Evaluation." *PVLDB*, 15(9), 1779–1797.
7. Liu, F.T., Ting, K.M., & Zhou, Z.-H. (2008). "Isolation Forest." *ICDM 2008*, 413–422.
8. Reimers, N. & Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." *EMNLP 2019*.
9. Burges, C.J.C. (2010). "From RankNet to LambdaRank to LambdaMART: An Overview." *Microsoft Research Technical Report MSR-TR-2010-82*.
10. Grover, A. & Leskovec, J. (2016). "node2vec: Scalable Feature Learning for Networks." *KDD 2016*, 855–864.
11. Hamilton, W.L., Ying, R., & Leskovec, J. (2017). "Inductive Representation Learning on Large Graphs." *NeurIPS 2017*.
12. Douze, M. et al. (2024). "The Faiss Library." *arXiv:2401.08281*.
13. Zhang, X. et al. (2024). "Efficient Neural Common Neighbor for Temporal Graph Link Prediction." *arXiv:2406.07926*.
14. Steiner, P. & Kaparthy, V. (2020). "Bilateral Trade Modelling with Graph Neural Networks." *PML4DC @ ICLR 2020*.
15. Broder, A. (2002). "A Taxonomy of Web Search." *ACM SIGIR Forum*, 36(2), 3–10.

High-Rigor Enhancement Proposals

Idea 1: Probabilistic Entity Resolution Pipeline for Company Name Canonicalization

CS Rigor Score: 9/10

- Algorithmic Complexity: 9/10
- Theoretical Depth: 9/10
- System Design: 8/10
- Novel Implementation: 9/10
- Learning Value: 10/10

Core Concept: The Transaction table stores `buyer` and `seller` as raw text. The same real-world company appears under dozens of spelling variations ("Muller & Phipps Pakistan Pvt Ltd", "M&P Pakistan", "MULLER AND PHIPPS"). This proposal implements a full Fellegi-Sunter probabilistic record linkage pipeline: blocking → pairwise comparison → probabilistic classification → transitive closure clustering → golden record creation. This directly addresses the #1 data quality problem in Zarailink and requires deep understanding of probabilistic models, string similarity algorithms, and graph clustering.

Technical Implementation:

1. **Blocking layer:** Implement Sorted Neighborhood blocking (Hernandez & Stolfo, 1995) using phonetic keys (Soundex/Metaphone of company names) to reduce $O(n^2)$ comparisons to $O(n \cdot w)$ where w is the window size. Evaluate multiple blocking strategies: standard blocking on first-3-characters, Sorted Neighborhood on Soundex codes, and canopy clustering using TF-IDF similarity thresholds.
2. **Comparison vector computation:** For each candidate pair, compute a feature vector of similarity scores:
 - Jaro-Winkler distance on full company name
 - Damerau-Levenshtein distance normalized
 - Soundex/Metaphone equality
 - Token-based Jaccard similarity (handles word reordering)
 - SBERT cosine similarity (reuse existing model)
 - Country match (binary)
 - Shared HS-code overlap ratio
3. **Fellegi-Sunter probabilistic classification:** Implement the core FS model — compute m-probabilities ($P(\text{agree} | \text{match})$) and u-probabilities ($P(\text{agree} | \text{non-match})$) using Expectation-Maximization (unsupervised, no labeled data required). Calculate composite match weights and classify pairs into Match/Non-Match/Possible-Match using optimal threshold selection.
4. **Transitive closure clustering:** Build a graph where nodes are company name variants and edges are classified matches. Apply connected components with quality refinement — implement the center clustering algorithm (Hassanzadeh et al., 2009) to avoid over-merging through long chains.
5. **Golden record creation:** For each cluster, select the canonical name using frequency-weighted voting with recency bias. Store the mapping in a new CompanyCanonical model linked to Transaction records.

Learning Outcomes:

- Probabilistic modeling (EM algorithm for unsupervised parameter estimation)
- String similarity algorithm internals (not just calling a library — implementing Jaro-Winkler from scratch)
- Blocking theory (understanding precision/recall tradeoffs in candidate generation)
- Graph clustering for entity resolution
- Evaluation metrics: pairwise precision/recall/F1, cluster purity, reduction ratio

Research Foundation:

1. Fellegi & Sunter (1969) — foundational probabilistic record linkage theory [Citation 1]
2. Binette & Steorts (2022) — comprehensive modern survey of entity resolution [Citation 2]
3. Hernandez & Stolfo (1995) — Sorted Neighborhood blocking method
4. Hassanzadeh et al. (2009) — clustering algorithms for entity resolution
5. Christen (2012) — "Data Matching" textbook (Springer), chapters on blocking and classification

Integration with Existing Architecture:

- New Django app: `entity_resolution`
- New models: `CompanyCanonical`, `NameVariant`, `ResolutionJob`
- Stage 3 (SupplierAggregator) aggregates by canonical company ID instead of raw text
- Node2Vec embeddings recomputed on the canonicalized graph
- Management command: `python manage.py resolve_entities`

Why This Has CS Rigor: Entity resolution requires implementing the EM algorithm, understanding the Fellegi-Sunter probabilistic framework's mathematical foundations (likelihood ratios, Neyman-Pearson lemma application), designing blocking strategies with formal analysis of pairs completeness vs reduction ratio, and handling transitive closure with quality guarantees. This cannot be trivially generated by AI because it requires understanding why each component exists theoretically and making design decisions based on the specific data characteristics (Pakistani company naming conventions, Urdu transliteration patterns, abbreviation norms).

Implementation Roadmap:

1. Week 1–2: Implement string similarity functions from scratch (Jaro-Winkler, Soundex, token Jaccard). Write unit tests.
2. Week 3: Implement blocking strategies. Benchmark pairs completeness and reduction ratio.
3. Week 4–5: Implement Fellegi-Sunter with EM. Manually label 200 pairs as ground truth for evaluation.
4. Week 6: Implement clustering and golden record creation. Evaluate end-to-end pipeline.
5. Week 7: Integrate with SupplierAggregator and retrain Node2Vec embeddings.

Estimated Effort: 7 weeks, High difficulty

Fallback Version: Skip the full EM-based Fellegi-Sunter model. Instead, implement deterministic matching with a hand-tuned composite score threshold. Still requires implementing blocking + string similarity + clustering, but avoids the probabilistic theory. Rigor score drops to ~6/10.

Idea 2: FAISS-Based Approximate Nearest Neighbor Index with Custom Quantization for Semantic Search

CS Rigor Score: 8/10

- Algorithmic Complexity: 9/10
- Theoretical Depth: 8/10
- System Design: 8/10
- Novel Implementation: 7/10
- Learning Value: 8/10

Core Concept: Stage 2 (QueryMatcher) currently loads a pickle file of SBERT embeddings and performs brute-force cosine similarity — $O(n)$ for every query. This proposal replaces it with a FAISS index using HNSW (Hierarchical Navigable Small World graphs), implements product quantization for memory compression, benchmarks recall@K against exact search, and adds a custom re-ranking layer. The student must understand the theory behind navigable small world graphs, skip-list-inspired hierarchical construction, and the recall/speed tradeoff space.

Technical Implementation:

1. **FAISS HNSW index construction:** Replace `search_index.pkl` with a FAISS `IndexHNSWFlat` index. Configure parameters: M (graph connectivity = 32), efConstruction (build-time search depth = 200), efSearch (query-time search depth = tunable).
2. **Product Quantization variant:** Implement `IndexIVFPQ` (Inverted File with Product Quantization) as an alternative. This requires understanding: Voronoi cell partitioning via k-means, vector decomposition into sub-vectors, codebook training per sub-space, and asymmetric distance computation. Benchmark against HNSW for this specific dataset.
3. **Recall@K evaluation framework:** Build a proper evaluation harness — compute exact neighbors (brute force), then measure recall@1, recall@5, recall@10 for HNSW and IVF-PQ at different parameter settings. Plot Pareto curves of recall vs QPS (queries per second).
4. **Hybrid re-ranking:** After FAISS returns top-50 approximate neighbors, apply exact cosine similarity re-ranking on this candidate set, then apply the existing keyword/fuzzy boosting. This two-stage retrieve-then-rerank pattern is standard in production IR systems.
5. **Dynamic index updates:** Implement an append-only FAISS index that can absorb new ProductSubCategory embeddings without full rebuild, using `index.add()` with an ID map.

Learning Outcomes:

- Approximate nearest neighbor theory (locality-sensitive hashing, graph-based search, quantization)
- HNSW algorithm internals (navigable small world graph construction, layered search)
- Product quantization (sub-vector decomposition, codebook training, asymmetric distance computation)
- IR evaluation methodology (recall@K, precision@K, MAP, NDCG)
- Speed/accuracy tradeoff analysis with empirical benchmarking

Research Foundation:

1. Malkov & Yashunin (2020) — HNSW paper [Citation 3]
2. Douze et al. (2024) — The Faiss Library [Citation 12]
3. Jégou, H., Douze, M., & Schmid, C. (2011). "Product Quantization for Nearest Neighbor Search." *IEEE TPAMI*, 33(1), 117–128.
4. Johnson, J., Douze, M., & Jégou, H. (2019). "Billion-Scale Similarity Search with GPUs." *IEEE Transactions on Big Data*, 7(3), 535–547.

Integration with Existing Architecture:

- Replace `search_index.pkl` with FAISS index file (`search_index.faiss`)
- Modify `QueryMatcher.__init__()` to load FAISS index instead of numpy array
- Modify `semantic_match()` to use `index.search(query_vector, k=50)` then re-rank
- Management command updated: `rebuild_search_index` outputs FAISS index
- Evaluation script: `evaluate_search_index.py` outputs recall/speed metrics

Why This Has CS Rigor: While FAISS is a library, understanding *why* HNSW works requires grasping the navigable small world phenomenon, the probabilistic analysis of search complexity in layered graphs ($O(\log n)$ expected time), and the role of parameter M in controlling the degree distribution. Product quantization requires understanding vector space decomposition, codebook distortion theory, and the asymmetric distance approximation bound. The evaluation framework requires formal IR metrics implementation.

Implementation Roadmap:

1. Week 1: Implement brute-force exact baseline with timing. Build evaluation dataset (queries with known relevant products).
2. Week 2: Implement FAISS HNSW index. Tune M , $efConstruction$, $efSearch$. Measure $recall@K$.
3. Week 3: Implement FAISS IVF-PQ. Tune $nlist$, $nprobe$, $code_size$. Compare to HNSW.
4. Week 4: Implement hybrid re-ranking pipeline. Plot Pareto curves. Write analysis.

Estimated Effort: 4 weeks, Medium-High difficulty

Fallback Version: Skip product quantization. Implement only HNSW with proper $recall@K$ evaluation and comparison to brute-force. Rigor score drops to $\sim 6/10$.

Idea 3: GNN-Based Link Prediction for Trade Partner Recommendation

CS Rigor Score: 9/10

- Algorithmic Complexity: 9/10
- Theoretical Depth: 9/10
- System Design: 8/10
- Novel Implementation: 9/10
- Learning Value: 10/10

Core Concept: The project currently has a "Link Prediction" page and Node2Vec embeddings, but no actual GNN or link prediction model. This proposal implements a proper Graph Convolutional Network (GCN) or GraphSAGE model for the buyer-seller trade graph, trained on a temporal link prediction task: given the trade network at time T , predict which new buyer-seller links will form at time $T+1$. This requires constructing a heterogeneous temporal graph, implementing message-passing layers, designing positive/negative edge sampling, and evaluating with AUC-ROC and Average Precision.

Technical Implementation:

1. **Temporal graph construction:** Build a time-stamped bipartite graph from Transaction records. Nodes = companies (buyers + sellers). Edges = trade relationships with attributes (volume, price, product, timestamp). Split into train/val/test by time: edges before T for training, edges in $[T, T+\Delta]$ for validation, edges after $T+\Delta$ for testing.
2. **Node feature engineering:** Each company node gets a feature vector combining: one-hot product category vector, total volume (log-normalized), trade partner count (degree), average price, country one-hot, and the existing Node2Vec embedding as a supplementary feature.
3. **GCN/GraphSAGE implementation:** Implement using PyTorch Geometric (PyG). Use 2–3 GraphSAGE convolution layers with mean aggregation. The message-passing update for node v at layer l is: $h_v^{(l)} = \sigma(W \cdot \text{CONCAT}(h_v^{(l-1)}, \text{AGG}(\{h_u^{(l-1)} : u \in N(v)\})))$. Implement both GCN (spectral) and GraphSAGE (sampling-based, inductive) and compare.
4. **Link prediction head:** For a candidate edge (u, v) , compute link score as: $\text{score}(u, v) = \text{MLP}(h_u \odot h_v)$ where $\odot$ is element-wise product. Train with binary cross-entropy loss on positive (real) and negative (sampled non-existent) edges. Use a 1:3 positive:negative sampling ratio with degree-biased negative sampling.
5. **Evaluation:** AUC-ROC, Average Precision (AP), Hits@10, Hits@50. Compare against baselines: common neighbors, Jaccard coefficient, Adamic-Adar, and the existing Node2Vec dot-product.

Learning Outcomes:

- Graph neural network theory (message passing, neighborhood aggregation, over-smoothing)
- Spectral vs spatial graph convolutions
- Temporal graph split methodology (avoiding data leakage)
- Negative edge sampling strategies
- Link prediction evaluation metrics
- PyTorch Geometric framework

Research Foundation:

1. Zhang & Chen (2018) — SEAL: Link Prediction Based on GNNs [Citation 4]
2. Kipf & Welling (2017) — Graph Convolutional Networks [Citation 5]
3. Hamilton, Ying & Leskovec (2017) — GraphSAGE [Citation 11]
4. Steiner & Kaparthy (2020) — Bilateral Trade Modelling with GNNs [Citation 14]
5. Zhang et al. (2024) — Temporal Neural Common Neighbor [Citation 13]

Integration with Existing Architecture:

- New module: `search/services/gnn_link_prediction.py`
- New model: `trade_data.LinkPredictionResult` to cache predictions
- Replaces the existing `ComparableFinder` with GNN-based similar company scoring
- Training script: `train_gnn.py` (separate from LTR training)
- Frontend: Updates Link Prediction page with actual model predictions + confidence scores

Why This Has CS Rigor: Implementing a GNN from scratch (even with PyG primitives) requires understanding the mathematical formulation of message passing, the spectral theory behind GCN (Chebyshev polynomial approximation of graph Laplacian filters), the inductive learning advantage of GraphSAGE, and the critical issue of temporal data leakage in link prediction evaluation. The student must make informed decisions about graph construction, feature engineering, layer depth (over-smoothing), and sampling strategy. This directly extends the existing (shallow) Node2Vec approach to a deep, learnable architecture.

Implementation Roadmap:

1. Week 1–2: Construct temporal bipartite graph from Transaction data. Implement train/val/test temporal splits. Compute baseline heuristic scores (CN, Jaccard, AA).
2. Week 3–4: Implement GraphSAGE with PyTorch Geometric. Train link prediction model with positive/negative sampling.
3. Week 5: Implement GCN variant. Compare architectures. Hyperparameter tuning.
4. Week 6: Evaluate all models. Integrate best model into the API endpoint. Update frontend.

Estimated Effort: 6 weeks, High difficulty

Fallback Version: Use only Node2Vec embeddings with a learned MLP classifier on top (skip the GNN). This still requires temporal split, negative sampling, and evaluation, but avoids implementing GNN layers. Rigor score drops to ~7/10.

Idea 4: Statistical Anomaly Detection Pipeline for Trade Price and Volume Data

CS Rigor Score: 8/10

- Algorithmic Complexity: 8/10
- Theoretical Depth: 8/10
- System Design: 7/10
- Novel Implementation: 8/10
- Learning Value: 8/10

Core Concept: Implement an anomaly detection system that identifies suspicious or noteworthy patterns in trade transaction data: price outliers (a shipment priced 3× the typical rate), volume spikes (sudden surge in a product's import volume), and structural anomalies (a company suddenly trading a product it has no history with). This directly feeds the Market Intelligence alerts system, transforming it from manually-created alerts to automated, data-driven intelligence.

Technical Implementation:

1. **Isolation Forest for multivariate anomalies:** Implement the Isolation Forest algorithm (Liu et al., 2008) from scratch — not just calling sklearn. Build isolation trees by randomly selecting features and split points, compute anomaly scores as the average path length across trees. Apply to feature vectors: [price_per_mt, volume_mt, shipment_count, days_since_last_trade] per (company, product, country) group.
2. **STL decomposition + statistical residual analysis:** For time-series price data per product, implement Seasonal-Trend decomposition using Loess (STL). Extract trend, seasonal, and residual components. Flag residual values exceeding 3σ as point anomalies. This catches price spikes that aren't explained by seasonal patterns.
3. **Z-score contextual anomaly detection:** For each transaction, compute a contextual Z-score relative to transactions of the same product in the same time window and country. Transactions with $|Z| > 3$ on price or volume are flagged. Implement Tukey's method as a fallback for non-Gaussian distributions.
4. **Structural novelty detection:** Flag transactions where a company trades a product for the first time, or trades with a new country for the first time. Score novelty as: $1 - (\text{historical_frequency} / \text{max_frequency})$. High novelty + high volume = alert candidate.
5. **Alert generation pipeline:** Aggregate detected anomalies into Alert objects with auto-generated headlines, severity scoring (based on anomaly score magnitude and business impact), and linked transactions. This replaces the manually created alerts in the market_intel app.

Learning Outcomes:

- Isolation Forest algorithm internals (random partitioning, path length scoring, contamination parameter)
- Time series decomposition theory (STL, seasonal adjustment)
- Statistical anomaly detection (Z-scores, Tukey fences, distributional assumptions)
- Multivariate vs univariate anomaly detection tradeoffs
- Evaluation of unsupervised anomaly detectors (synthetic injection methodology)

Research Foundation:

1. Liu, Ting & Zhou (2008) — Isolation Forest [Citation 7]
2. Schmidl et al. (2022) — Comprehensive evaluation of time series anomaly detectors [Citation 6]
3. Cleveland, R.B. et al. (1990). "STL: A Seasonal-Trend Decomposition Procedure Based on Loess." *Journal of Official Statistics*, 6(1), 3–73.
4. Chandola, V., Banerjee, A. & Kumar, V. (2009). "Anomaly Detection: A Survey." *ACM Computing Surveys*, 41(3), 1–58.

Integration with Existing Architecture:

- New module: `market_intel/services/anomaly_detector.py`
- Hooks into existing `Alert` model — detected anomalies become new alerts automatically
- Management command: `python manage.py detect_anomalies --window 30d`
- Optionally triggered on data ingestion (after `ingest_transactions`)
- Dashboard widget showing anomaly count by severity

Why This Has CS Rigor: Implementing Isolation Forest from scratch (building random trees, computing average path length, understanding the connection to binary search tree theory) requires algorithmic thinking that goes beyond library usage. STL decomposition requires understanding LOESS regression. The evaluation challenge of unsupervised anomaly detection (no ground truth labels) requires the student to design a synthetic anomaly injection experiment and measure detection rates — a legitimate research methodology.

Implementation Roadmap:

1. Week 1–2: Implement Isolation Forest from scratch. Unit test against sklearn's implementation to verify correctness.
2. Week 3: Implement STL decomposition and residual analysis for price time series.
3. Week 4: Implement contextual Z-score detection and structural novelty scoring.
4. Week 5: Build alert generation pipeline. Integrate with market_intel Alert model.
5. Week 6: Design synthetic anomaly injection experiment. Evaluate all detectors. Write analysis.

Estimated Effort: 6 weeks, Medium-High difficulty

Fallback Version: Use only Z-score and Tukey's method for anomaly detection (skip Isolation Forest from-scratch implementation and STL). Still requires integration and evaluation, but significantly less algorithmic depth. Rigor score drops to ~5/10.

Idea 5: Fine-Tuned Domain-Specific SBERT with Contrastive Learning for Agricultural Trade Queries

CS Rigor Score: 8/10

- Algorithmic Complexity: 7/10
- Theoretical Depth: 8/10
- System Design: 7/10
- Novel Implementation: 8/10
- Learning Value: 9/10

Core Concept: The current system uses `all-MiniLM-L6-v2` as a pretrained black box. Agricultural trade terminology has domain-specific semantics that a general-purpose model may not capture well ("IRRI-6" is a rice variety, "1006.30" is its HS code, "broken rice" $\neq$ "rice bran"). This proposal fine-tunes the SBERT model on domain-specific data using contrastive learning (Multiple Negatives Ranking Loss), creating training pairs from the product taxonomy and transaction co-occurrence data.

Technical Implementation:

1. **Training data construction from product taxonomy:** Generate positive pairs from the existing 4-level product hierarchy:
 - (ProductItem.name, ProductSubCategory.name) — e.g., ("Dextrose Monohydrate", "Dextrose")
 - (ProductSubCategory.name, ProductCategory.name) — e.g., ("Dextrose", "Sugar Derivatives")
 - Co-traded products from transactions (products commonly shipped together form soft positives)
2. **Hard negative mining:** For each positive pair, select hard negatives — products in different categories but with superficially similar names or overlapping keywords. E.g., "Rice Bran Oil" vs "Rice" (different product, similar text).
3. **Contrastive training with MNR Loss:** Fine-tune the SBERT model using Multiple Negatives Ranking (MNR) loss. For a batch of (anchor, positive) pairs, all other positives in the batch serve as in-batch negatives. This is the standard SentenceTransformer fine-tuning approach but applied to a novel domain-specific dataset.
4. **Evaluation:** Compare before/after fine-tuning using:
 - Retrieval accuracy on a held-out set of (query, relevant_product) pairs
 - Spearman correlation of predicted similarity vs human-annotated similarity
 - Embedding quality visualization (t-SNE/UMAP of product embeddings before/after)

Learning Outcomes:

- Transfer learning and domain adaptation theory
- Contrastive learning (triplet loss, MNR loss, in-batch negatives)
- Training data engineering for NLP (positive pair construction, hard negative mining)
- Embedding evaluation methodology
- SentenceTransformers training pipeline internals

Research Foundation:

1. Reimers & Gurevych (2019) — Sentence-BERT [Citation 8]
2. Henderson, M. et al. (2017). "Efficient Natural Language Response Suggestion for Smart Reply." *arXiv:1705.00652*.
3. Gao, T., Yao, X. & Chen, D. (2021). "SimCSE: Simple Contrastive Learning of Sentence Embeddings." *EMNLP 2021*.

Integration with Existing Architecture:

- Fine-tuned model replaces `all-MiniLM-L6-v2` in `QueryMatcher`
- Training script: `fine_tune_sbert.py`
- Rebuild `search_index.pkl` / FAISS index with new embeddings
- A/B evaluation comparing original vs fine-tuned model on search quality

Why This Has CS Rigor: Fine-tuning requires understanding the SBERT architecture (Siamese network with mean pooling), the mathematical formulation of contrastive loss functions, and the critical challenge of training data construction. The student must reason about what makes a "good" training pair in the agricultural trade domain — this is domain-specific feature engineering that cannot be trivially automated.

Implementation Roadmap:

1. Week 1: Construct training dataset from product taxonomy. Manual annotation of 100 query-product pairs for evaluation.
2. Week 2: Implement fine-tuning pipeline with MNR loss. Train model.
3. Week 3: Hard negative mining and re-training. Evaluate against baseline.
4. Week 4: Integration, index rebuild, A/B comparison.

Estimated Effort: 4 weeks, Medium difficulty

Fallback Version: Instead of fine-tuning, implement a learned query expansion module that uses the product taxonomy to expand search terms before embedding. Less rigorous but still demonstrates domain adaptation thinking. Rigor score drops to ~5/10.

Idea 6: Formal Query Language with PEG Grammar and Learned Intent Classifier

CS Rigor Score: 8/10

- Algorithmic Complexity: 8/10
- Theoretical Depth: 9/10
- System Design: 7/10
- Novel Implementation: 8/10
- Learning Value: 9/10

Core Concept: Stage 1 (QueryInterpreter) is currently a large regex-based parser with keyword counting for intent detection. This proposal replaces it with two components: (a) a Parsing Expression Grammar (PEG) for structured query syntax, giving the system a formal, unambiguous grammar; and (b) a trained BERT-based intent classifier for natural language queries that maps to one of the 9 query families. The PEG parser handles structured queries; the intent classifier handles free-form queries; and a routing layer decides which to invoke.

Technical Implementation:

1. **PEG grammar definition:** Define a formal grammar for structured trade queries using a PEG parser (e.g., `parsimonious` or `pypeg2` library):

```
Query    ← Intent Product Filters*
Intent   ← 'find' / 'show' / 'compare' / 'who'
Product  ← ProductName ('from' Country)?
Filters  ← PriceFilter / VolumeFilter / TimeFilter
PriceFilter ← ('under' / 'above' / 'between') Price
```

This provides formal parsing with clear precedence rules, composability, and error recovery.

2. **Intent classifier (BERT-based):** Fine-tune a small BERT model (e.g., `prajjwal1/bert-tiny`) or DistilBERT) on a labeled dataset of trade queries mapped to family F1–F9. Training data generated by template-based augmentation + manual annotation of 500 real queries. Evaluate with macro F1 across all 9 families.
3. **Routing layer:** If the query matches the PEG grammar → use structured parser (high confidence). If not → fall back to the intent classifier → map predicted family + extracted entities to a `ParsedQuery` object. This two-tier approach gives both precision (structured) and recall (ML fallback).
4. **Named Entity Recognition for trade queries:** Train a lightweight NER model (or rule-based with learned confidence) to extract: PRODUCT, COUNTRY, COMPANY, QUANTITY, PRICE, TIME entities from natural language queries. Feed extracted entities into the `ParsedQuery` constructor.

Learning Outcomes:

- Formal language theory (PEG grammars, packrat parsing, comparison with CFGs)
- Transfer learning for text classification (BERT fine-tuning on small datasets)
- NER model training and evaluation (entity-level F1, BIO tagging)
- Hybrid system design (rule-based + ML with confidence-based routing)
- Evaluation methodology for multi-component NLP systems

Research Foundation:

1. Ford, B. (2004). "Parsing Expression Grammars: A Recognition-Based Syntactic Foundation." *POPL 2004*, 111–122.
2. Broder (2002) — Taxonomy of web search queries [Citation 15]
3. Devlin, J. et al. (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." *NAACL 2019*.
4. Chandrakala, C.B. et al. (2024). "An intent recognition pipeline for conversational AI." *IJIT*, 16, 731–743.

Integration with Existing Architecture:

- Replaces `query_parser.py` with `query_parser_v2.py`
- PEG grammar defined in `grammar.peg` file
- Intent classifier model stored alongside LTR model
- ParsedQuery output format unchanged — downstream stages (2–4) unaffected
- Evaluation script compares v1 (regex) vs v2 (PEG+BERT) on a test set

Why This Has CS Rigor: Writing a PEG grammar requires understanding formal language theory — the relationship between PEGs and context-free grammars, the ordered choice operator, the guarantee of unambiguous parsing, and the packrat parsing algorithm ($O(n)$ with memoization). Fine-tuning BERT for intent classification on a small domain-specific dataset requires understanding attention mechanisms, tokenization, and few-shot learning challenges. The routing layer requires principled confidence calibration.

Implementation Roadmap:

1. Week 1: Define PEG grammar. Implement parser. Test on 100 structured queries.
2. Week 2: Create training dataset for intent classifier (template augmentation + manual annotation).
3. Week 3: Fine-tune DistilBERT intent classifier. Evaluate macro F1.
4. Week 4: Implement routing layer and NER. End-to-end integration test.
5. Week 5: Comparative evaluation against regex-based v1.

Estimated Effort: 5 weeks, Medium-High difficulty

Fallback Version: Implement only the PEG grammar (skip the BERT classifier). Use the existing keyword-counting for intent detection but replace entity extraction with the formal grammar. Rigor score drops to ~6/10.

Idea 7: Online Learning-to-Rank with Click-Through Feedback Integration

CS Rigor Score: 7/10

- Algorithmic Complexity: 7/10
- Theoretical Depth: 8/10
- System Design: 7/10
- Novel Implementation: 7/10
- Learning Value: 8/10

Core Concept: The LTR model (Stage 4) is currently trained once on pseudo-labels. This proposal implements a proper online learning loop: log user interactions (clicks, contact unlocks, time-spent) as implicit relevance signals, construct LTR training pairs with position-bias correction, retrain the model on a schedule, and evaluate improvement using held-out NDCG.

Technical Implementation:

1. **Implicit feedback collection:** Extend `UserInteraction` to log: search query, result list (ordered), which results were clicked, click position, time-to-click, whether a contact was unlocked, and session duration on the supplier detail page. Each search-session becomes one LTR query group.
2. **Position bias correction:** Implement a simple inverse-propensity-scoring (IPS) model: the probability of a result being examined decreases with rank position. Estimate examination probabilities from aggregate click data using the cascade model (Craswell et al., 2008). Adjust click labels by $1/P(\text{examine} \mid \text{position})$ before training.
3. **Pairwise label construction:** For each query group, construct pairwise preferences: clicked result > unclicked result at higher position (with position bias correction). Feed into LightGBM LambdaRank with the corrected labels.
4. **Offline evaluation:** Implement NDCG@5, NDCG@10, MAP@10 evaluation on held-out query sessions. Compare: pseudo-label model vs click-trained model vs heuristic-only baseline.
5. **Periodic retraining pipeline:** Management command that retrains the LTR model weekly using accumulated click data. Version the model files. A/B test infrastructure to compare model versions.

Learning Outcomes:

- Learning-to-Rank theory (pointwise, pairwise, listwise)
- Position bias in implicit feedback (examination hypothesis, IPS correction)
- NDCG metric and its properties (logarithmic discount, ideal ranking)
- Online learning and model versioning
- A/B testing methodology for IR systems

Research Foundation:

1. Burges (2010) — From RankNet to LambdaMART overview [Citation 9]
2. Craswell, N. et al. (2008). "An experimental comparison of click position-bias models." *WSDM 2008*, 87–94.
3. Joachims, T. et al. (2017). "Unbiased Learning-to-Rank with Biased Feedback." *WSDM 2017*, 781–789.
4. Ai, Q. et al. (2018). "Unbiased Learning to Rank with Unbiased Propensity Estimation." *SIGIR 2018*, 385–394.

Integration with Existing Architecture:

- Extends existing `UserInteraction` model with search-result-list fields
- Extends existing `ranking_ltr.py` with click-based training
- New management command: `retrain_ltr --source clicks`
- Model versioning: `lgbm_ltr_v{N}.txt`

Why This Has CS Rigor: Position bias correction is a non-trivial statistical problem — the student must understand the examination hypothesis, implement propensity estimation, and apply inverse propensity scoring. NDCG implementation requires understanding the mathematical properties of the metric (discounted cumulative gain, ideal ranking computation). The online learning loop introduces model versioning and evaluation challenges.

Estimated Effort: 5 weeks, Medium difficulty

Fallback Version: Skip position bias correction. Use raw click counts as relevance labels. Still requires the feedback loop, retraining pipeline, and NDCG evaluation. Rigor score drops to ~5/10.

Recommended Implementation Path

Priority 1: Entity Resolution Pipeline (Idea 1)

Justification: This addresses the most fundamental data quality problem in the system. Every downstream component (search aggregation, graph embeddings, company profiles, trade ledger analytics) is degraded by the unnormalized company names. Fixing this has the highest impact-to-effort ratio and produces the most academically defensible contribution — it touches probabilistic theory, string algorithms, graph clustering, and systems design.

Prerequisites:

- Study the Fellegi-Sunter framework (read Binette & Steorts 2022 survey)
- Understand EM algorithm (Bishop PRML Chapter 9)
- Implement Jaro-Winkler and Soundex from scratch (data structures knowledge)

Timeline:

- Week 1–2: String similarity algorithms + blocking strategies
- Week 3–4: Fellegi-Sunter with EM + evaluation dataset
- Week 5–6: Clustering + golden record + integration
- Week 7: End-to-end evaluation + write-up

Priority 2: GNN-Based Link Prediction (Idea 3)

Justification: This transforms the existing (shallow) Node2Vec approach into a proper deep graph learning system. It's the most research-aligned enhancement and produces the strongest academic narrative: "We extended a trade network embedding system from shallow (Node2Vec) to deep (GNN) methods and demonstrated improved link prediction performance." The temporal evaluation methodology is rigorous and publishable.

Prerequisites:

- Complete PyTorch Geometric tutorial
- Study GCN and GraphSAGE papers
- Understand message passing framework

Timeline:

- Week 1–2: Graph construction + temporal splits + baselines
- Week 3–4: GCN and GraphSAGE implementation
- Week 5: Evaluation + comparison
- Week 6: Integration + frontend updates

Priority 3: FAISS ANN Index (Idea 2)

Justification: This is the most tightly scoped enhancement with the clearest before/after evaluation. It directly improves a measurable system property (search latency) while requiring understanding of fundamental data structure theory (skip lists, proximity graphs, Voronoi partitioning). It complements the entity resolution work (canonicalized company names → better embeddings → better ANN search).

Prerequisites:

- Read HNSW paper (Malkov & Yashunin 2020)
- Understand FAISS API
- Build evaluation framework

Timeline:

- Week 1: Baseline + evaluation harness
- Week 2: HNSW implementation + tuning
- Week 3: IVF-PQ comparison
- Week 4: Integration + analysis

Conclusion

Zarailink already has a strong engineering foundation. The proposed enhancements are designed to transform it from a well-built product into an academically rigorous system that demonstrates genuine CS depth.

The three recommended priorities — Entity Resolution, GNN Link Prediction, and FAISS ANN Indexing — form a coherent narrative: "We built an agricultural trade intelligence platform and then systematically improved its core data quality (entity resolution), its search infrastructure (ANN indexing), and its recommendation capability (GNN link prediction), evaluating each enhancement with formal metrics from the information retrieval and machine learning literature."

This narrative is compelling for a final-year evaluation because each component is independently evaluable, builds on theoretical foundations, requires non-trivial implementation, and produces measurable improvements on the existing system. The total scope of all three priorities is approximately 17 weeks — feasible within a two-semester final year project.